Università degli Studi di Messina

CdL in Scienze Informatiche
Sicurezza dei Sistemi Software
A.A. 2024/2025

Progetto "Fort Knox": Implementazione di un'Infrastruttura Bancaria Sicura e Virtualizzata

Sandi Russo
Matr. 553675

1 febbraio 2026

Sommario

La presente relazione illustra in dettaglio la progettazione, l'implementazione e la verifica di "Fort Knox", un prototipo di sistema bancario ad alta sicurezza. Il progetto nasce dall'esigenza di applicare concretamente i principi della *Security by Design*, integrando competenze trasversali che spaziano dalla sicurezza delle reti alla crittografia applicata, fino allo sviluppo full-stack moderno.

L'infrastruttura realizzata simula un ambiente di produzione reale basato su una rigorosa segmentazione di rete. Il perimetro è difeso da un firewall **pfSense** virtualizzato, che isola una *Demilitarized Zone* (DMZ) contenente i servizi applicativi dalla rete pubblica. L'architettura software adotta un approccio a microservizi containerizzati tramite **Docker**, orchestrati per garantire il principio del minimo privilegio.

Particolare enfasi è stata posta sulla protezione del dato: sono stati implementati algoritmi di crittografia simmetrica **AES-256** per l'archiviazione sicura dei documenti e crittografia asimmetrica **RSA-2048** per garantire il non ripudio delle transazioni finanziarie. Il risultato è un sistema resiliente, capace di mitigare le più comuni vulnerabilità web (OWASP Top 10) e di proteggere la riservatezza degli utenti anche in caso di compromissione fisica del server.

Indice

1 Introduzione e Obiettivi **1**
1.1 Stack Tecnologico 1

2 Architettura di Rete e Sicurezza Perimetrale **1**
2.1 Configurazione del Firewall pfSense 2
2.2 Regole di Accesso e Port Forwarding (NAT) 2

3 Architettura Applicativa a Microservizi **2**
3.1 Definizione dei Container 3

4 Implementazione della Sicurezza Software **3**
4.1 Crittografia Simmetrica (AES-256) per i Documenti 3
4.2 Crittografia Asimmetrica (RSA) e Firma Digitale 4
4.2.1 Gestione delle Chiavi 4
4.2.2 Firma della Transazione 4
4.3 Autenticazione Stateless con JWT 5

5 Verifica Sperimentale **5**
5.1 Verifica della Cifratura a Riposo 5
5.2 Test Funzionale Completo 5

6 Conclusioni **6**
6.1 Sviluppi Futuri 6

1 Introduzione e Obiettivi

Nel panorama attuale della sicurezza informatica, la protezione dei dati finanziari e personali rappresenta una delle sfide più critiche. Il progetto "Fort Knox" si prefigge l'obiettivo di costruire una piattaforma bancaria simulata che non si limiti alla funzionalità, ma che ponga la sicurezza come requisito fondante dell'architettura.

Gli obiettivi principali del progetto sono stati:

- **Difesa in Profondità (Defense in Depth):** Implementare molteplici livelli di sicurezza (rete, applicazione, dati) in modo che il fallimento di un livello non comprometta l'intero sistema.
- **Protezione dei Dati a Riposo:** Garantire che i documenti sensibili degli utenti siano illeggibili se sottratti direttamente dal disco rigido.
- **Integrità delle Transazioni:** Utilizzare la firma digitale per impedire la falsificazione o il ripudio dei bonifici bancari.
- **Isolamento dei Servizi:** Utilizzare la containerizzazione per limitare la superficie di attacco ("Blast Radius") in caso di compromissione di un singolo servizio.

1.1 Stack Tecnologico

La scelta delle tecnologie è stata guidata da criteri di robustezza, supporto della community e aderenza agli standard industriali:

- **Virtualizzazione:** Oracle VirtualBox per l'hosting delle VM.
- **Sicurezza di Rete:** pfSense (Firewall/Router basato su FreeBSD).
- **Containerizzazione:** Docker e Docker Compose per l'orchestrazione dei microservizi.
- **Backend:** Python FastAPI, scelto per le prestazioni elevate e le robuste librerie crittografiche (`cryptography`, `passlib`).
- **Frontend:** React con TypeScript e Ant Design per un'interfaccia utente moderna e tipizzata.
- **Database:** PostgreSQL 15, un RDBMS solido e conforme ACID.

2 Architettura di Rete e Sicurezza Perimetrale

La base della sicurezza di "Fort Knox" risiede nella topologia di rete. Non è stato utilizzato un semplice bridging delle interfacce, ma è stata costruita una rete segmentata gestita da un firewall dedicato.

2.1 Configurazione del Firewall pfSense

È stata installata e configurata una macchina virtuale **pfSense** che agisce come gateway unico per l'intera infrastruttura. La configurazione delle interfacce è la seguente:

1. **WAN (Interfaccia em0):** Configurata in modalità *NAT* verso l'host fisico. Questa interfaccia simula la connessione a Internet pubblica e acquisisce un indirizzo IP dinamico (es. `10.0.2.15`) dalla rete di VirtualBox.

2. **LAN/DMZ (Interfaccia vtnet0):** Configurata su una *Rete Interna* virtuale isolata. Su questa interfaccia, pfSense agisce come server DHCP e gateway, gestendo la sottorete `172.16.0.0/24`. L'indirizzo del gateway è `172.16.0.1`.

La macchina virtuale server (Ubuntu Server 22.04), che ospita l'applicazione Docker, è connessa esclusivamente all'interfaccia LAN (IP statico `172.16.0.15`). Di conseguenza, questa macchina è **invisibile e irraggiungibile** direttamente da Internet, proteggendola da scansioni di porte e attacchi diretti.

2.2 Regole di Accesso e Port Forwarding (NAT)

Per consentire l'utilizzo dei servizi mantenendo l'isolamento, è stata adottata una politica di firewall "Default Deny" (nega tutto tranne ciò che è esplicitamente permesso). L'accesso ai servizi avviene esclusivamente tramite **Port Forwarding (DNAT)**.

Di seguito la tabella delle regole NAT configurate su pfSense:

Tabella 1: Regole di Port Forwarding su pfSense

Servizio	**Descrizione**	**Porta WAN**	**Destinazione LAN**
SSH Tunnel	Accesso amministrativo cifrato al server	2222	172.16.0.15:22
Frontend HTTPS	Interfaccia Web utente (React/Nginx)	443	172.16.0.15:5173
Backend API	API REST per la logica di business	8001	172.16.0.15:8001

Questa configurazione garantisce che un attaccante esterno veda aperte solo le porte strettamente necessarie sul firewall pubblico, mentre la struttura interna della rete rimane offuscata.

3 Architettura Applicativa a Microservizi

L'applicazione non è un monolite, ma è stata decomposta in microservizi containerizzati gestiti da **Docker Compose**. Questo approccio offre vantaggi significativi in termini di sicurezza: ogni servizio gira in un ambiente isolato (sandbox) con accesso limitato alle risorse del sistema operativo host.

3.1 Definizione dei Container

Il file `docker-compose.yml` definisce l'orchestrazione dei tre componenti principali, collegati tramite una rete bridge interna (`fort_knox_net`) che non espone porte all'esterno se non quelle mappate esplicitamente.

```yaml
services:
  # 1. DATABASE: Accessibile SOLO dagli altri container, mai dall'
    esterno
  db:
    image: postgres:15
    container_name: fort_knox_db
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    networks:
      - fort_knox_net

  # 2. BACKEND: Contiene la logica crittografica
  backend:
    build: ./backend
    depends_on:
      - db
    volumes:
      # Montaggio del volume per la persistenza sicura dei file cifrati
      - ./backend/safe_storage:/app/safe_storage
    networks:
      - fort_knox_net

  # 3. FRONTEND: Server Nginx per servire l'app React
  frontend:
    build: ./frontend
    ports:
      - "5173:443" # Esposizione tramite tunnel SSL
    networks:
      - fort_knox_net
```

Listing 1: Configurazione dei servizi Docker

4 Implementazione della Sicurezza Software

Questo capitolo descrive il cuore del progetto: le misure crittografiche e logiche implementate per proteggere i dati e le transazioni.

4.1 Crittografia Simmetrica (AES-256) per i Documenti

Uno dei requisiti critici era la protezione dei documenti di identità caricati dagli utenti. Per garantire la riservatezza (*Confidentiality*), è stato implementato l'algoritmo **AES (Advanced Encryption Standard)** con chiave a 256 bit in modalità **CBC (Cipher Block Chaining)**.

Il flusso di sicurezza è il seguente:

1. L'utente carica il file tramite interfaccia web sicura.

2. Il backend riceve il file stream in memoria (senza scriverlo su disco).
3. Viene generato un *Salt* casuale e un *Initialization Vector (IV)* unico per ogni file.
4. Il contenuto viene cifrato.
5. Il file cifrato viene salvato su disco con estensione `.enc`.

Questo approccio garantisce che, anche in caso di furto fisico del server, i file siano matematicamente impossibili da leggere senza la chiave di sistema.

```python
def encrypt_file_content(file_bytes, password):
    key_length = 32 # AES-256
    bs = AES.block_size
    salt = urandom(bs)
    # Derivazione della chiave sicura (PBKDF2-like)
    key, iv = derive_key_and_iv(password, salt, key_length, bs)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    # Padding PKCS7 necessario per blocchi non allineati
    padding_length = (bs - len(file_bytes) % bs) or bs
    file_bytes += (padding_length * chr(padding_length)).encode()
    return salt + cipher.encrypt(file_bytes)
```

Listing 2: Snippet della funzione di cifratura AES

4.2 Crittografia Asimmetrica (RSA) e Firma Digitale

Per garantire il **Non Ripudio** e l'**Integrità** delle transazioni bancarie (bonifici), è stata utilizzata la crittografia asimmetrica RSA.

4.2.1 Gestione delle Chiavi

Al momento della registrazione dell'utente:

1. Viene generata una coppia di chiavi RSA a 2048 bit.
2. La **Chiave Pubblica** viene salvata in chiaro nel database (visibile per verifica).
3. La **Chiave Privata** viene cifrata con AES (usando una Master Key di sistema) prima di essere salvata nel database.

Questa tecnica, nota come *Key Wrapping*, assicura che la chiave privata non risieda mai in chiaro nel database.

4.2.2 Firma della Transazione

Quando un utente esegue un bonifico:

1. Il backend decifra temporaneamente la chiave privata dell'utente in memoria RAM.
2. Viene creato un *hash* (SHA-256) contenente i dati della transazione (Mittente, Destinatario, Importo, Timestamp).

3. L'hash viene firmato con la chiave privata.
4. La firma viene allegata alla transazione.

In questo modo, qualsiasi modifica successiva all'importo o al destinatario invaliderebbe la firma matematica.

4.3 Autenticazione Stateless con JWT

Il sistema abbandona le vecchie sessioni server-side in favore dello standard **JWT (JSON Web Token)**.

- **Login:** L'utente invia le credenziali. La password viene verificata confrontando l'hash **Bcrypt** salvato nel DB.
- **Token Issue:** Se valida, il server firma un token JWT contenente l'identità dell'utente e una scadenza.
- **Richieste Successive:** Il Frontend allega il token nell'header `Authorization: Bearer`. Un middleware nel backend verifica la firma del token prima di processare qualsiasi richiesta (es. download file o bonifico).

5 Verifica Sperimentale

Il sistema è stato sottoposto a test per validare l'efficacia delle misure di sicurezza.

5.1 Verifica della Cifratura a Riposo

Per dimostrare che i dati sensibili non sono esposti, è stato eseguito un accesso diretto al database PostgreSQL bypassando l'applicazione.

Eseguendo la query direttamente nel container DB:

```
SELECT email, balance, encrypted_private_key FROM users;
```

Il risultato mostra che la colonna della chiave privata contiene una stringa Base64 incomprensibile (il ciphertext AES), confermando che un amministratore di database malintenzionato non potrebbe impersonare gli utenti per firmare transazioni a loro nome.

5.2 Test Funzionale Completo

È stato simulato un intero ciclo operativo:

1. **Registrazione:** Creazione utente con upload di un documento PDF. Verifica della creazione del file `.enc` nella cartella `safe_storage`.
2. **Accesso:** Login corretto e ottenimento del Token JWT.
3. **Operatività:** Esecuzione di un bonifico. Verifica nei log del backend della generazione e validazione della firma RSA.
4. **Recupero Dati:** Download del documento dalla dashboard. Il sistema ha decifrato correttamente il file al volo, restituendo il PDF originale.

6 Conclusioni

Il progetto "Fort Knox" ha permesso di realizzare un prototipo funzionale che rispetta i moderni standard di sicurezza. L'architettura implementata dimostra come la combinazione di **segmentazione di rete** (pfSense), **isolamento dei processi** (Docker) e **crittografia applicativa robusta** (AES/RSA) possa mitigare efficacemente i rischi di attacchi esterni e interni.

6.1 Sviluppi Futuri

Per evolvere il prototipo verso un sistema di produzione, si propongono i seguenti miglioramenti:

- **HSM (Hardware Security Module):** Spostare la gestione delle chiavi private RSA dal database software a un modulo hardware dedicato.
- **Kubernetes:** Migrazione a un orchestratore K8s per gestire la scalabilità e l'alta disponibilità dei container.
- **VPN Site-to-Site:** Sostituire il Port Forwarding amministrativo con una VPN WireGuard gestita direttamente da pfSense.
- **CI/CD Security:** Implementazione di pipeline automatiche per la scansione statica del codice (SAST) alla ricerca di vulnerabilità.